# DRONE ASSEMBLY LINE — LOGGING & METRICS SYSTEM

SYSC 3303A W2026 • Dr. Rami Sabouni • Assignment 4

---

## 1) Overview

Agent places two components; each Technician owns one component and assembles a drone with the other two. **AssemblyTable** (monitor) synchronizes `wait()` / `notifyAll()`. **EventLogger** buffers events and flushes via a daemon thread to a run-ID log. **LogAnalyzer** computes throughput, utilization, and per-thread response times from the log.

### Core Events

- `WAIT_START` / `WAIT_END` — emitted at real monitor waits.
- `PLACED_COMPONENTS`, `PICKED_UP`, `DRONE_ASSEMBLED`.
- `TABLE_FULL`, `TABLE_EMPTY`.
- `WORK_START` / `WORK_END`, `RESPONSE_TIME`.
- `SYSTEM_START` / `SYSTEM_END`, `JOB_COMPLETED`.

### Log Line Format

```
Event log: [yyyy-MM-dd HH:mm:ss.SSS , ENTITY , EVENT_CODE , key=value; ...]
```

Keys typically include `components=[...,...]`, `drones`, and `duration`.

---

## 2) Build & Run

1. Open in IntelliJ → ensure SDK is Java 8+.
2. Run `AssemblyTable.main()`.
3. At shutdown, `LogAnalyzer.main()` runs automatically and writes `metrics.txt`.

### Config

- **Max drones**: `AssemblyTable.getMaxDrones()`
- **Log file**: `EventLogger.setLogFileName(...)`
- **Flush interval**: `EventLogger.setFlushIntervalMs(...)`

---

## 3) Diagrams

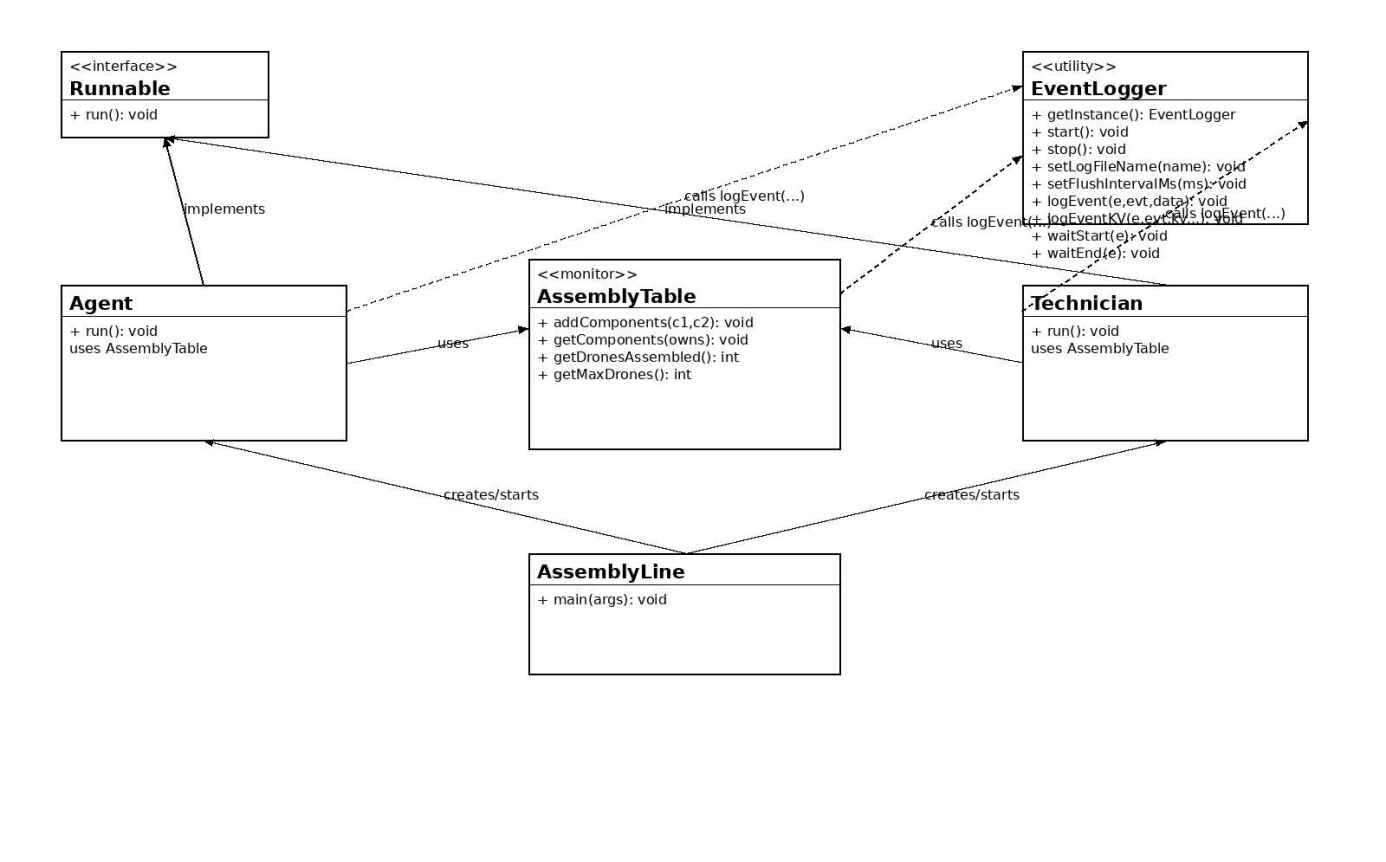



Figure 1 — UML Class Diagram (Agent, Technician, AssemblyTable monitor, EventLogger, AssemblyLine)

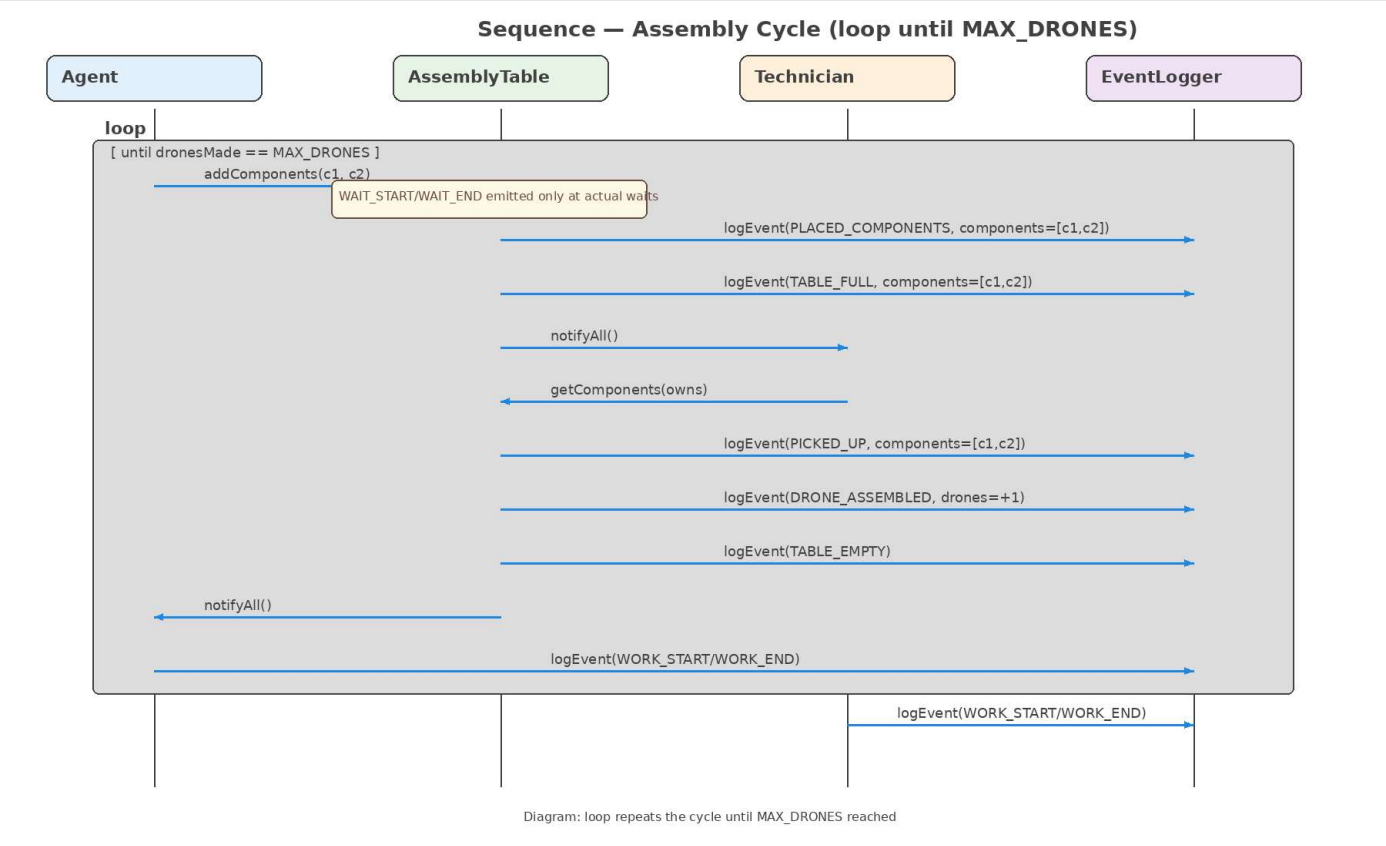
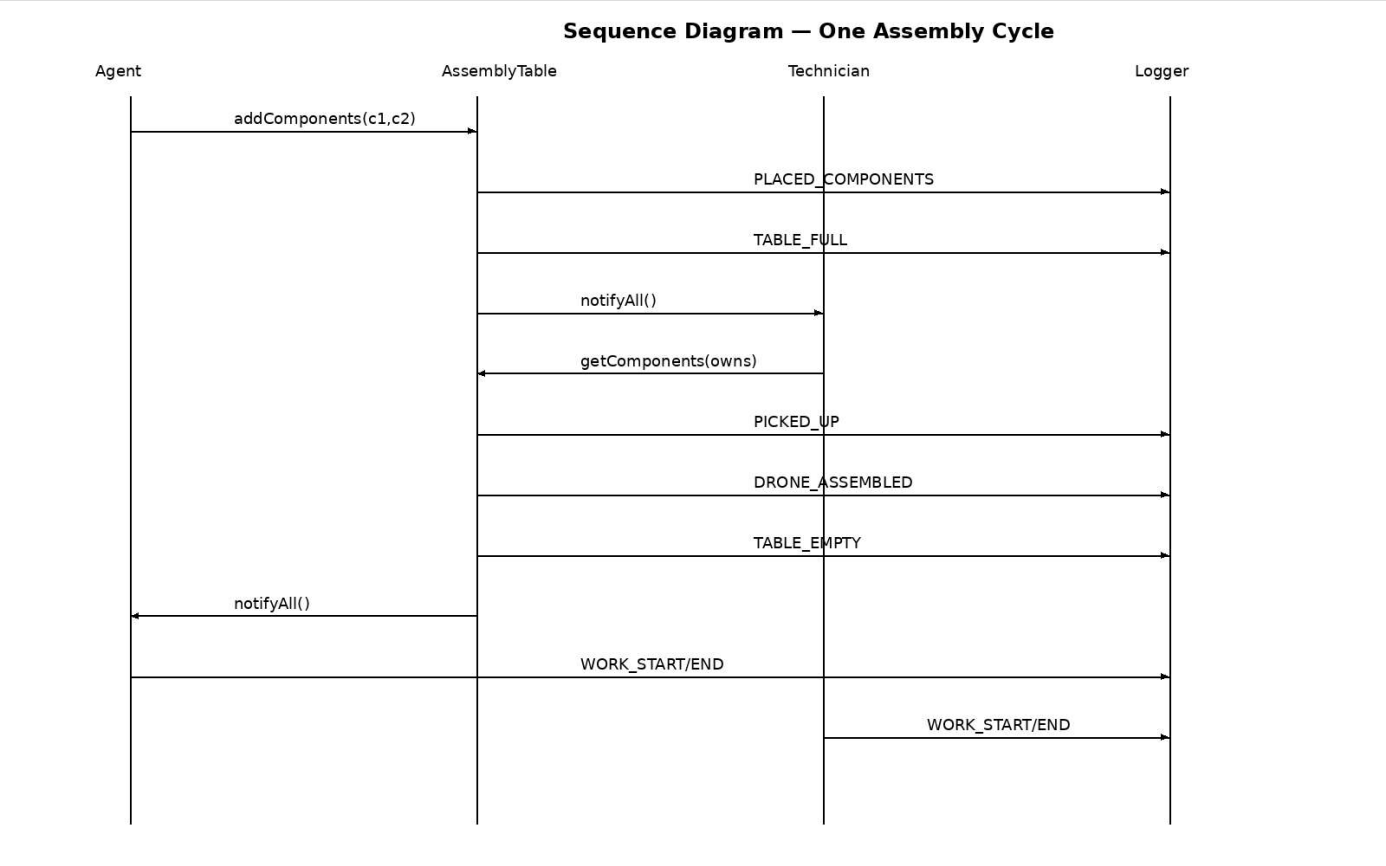



Figure 2 — Sequence Diagram (loop until MAX_DRONES)




Appendix — Sequence Diagram (single cycle)

---

# 4) Output

- `assembly_log_YYYYMMDD_HHMMSS.txt` — run-ID log
- `metrics.txt` — analyzer summary (throughput, utilization, response)